Session 14 Slides

Quantum Convolutional Neural Networks (QCNNs)

Session road-map

- Recap: why hybrid methods first
- What is a QCNN? (quantum analogue of conv + pool)
- Quanvolutional filters (patch → encoding → PQC → features)
- Parameter sharing & translation equivariance
- Pooling on quantum circuits (measurement / unitary pooling)
- Architectures & training loops (optimisers, losses)
- Qiskit ML implementations (EstimatorQNN / SamplerQNN + Torch)
- Evaluation vs small classical CNNs
- Practical tips, pitfalls, and lab outline
- Mini-exercises + answers

0) Recap — Hybrid methods → QCNNs

- Hybrid pattern: keep quantum circuits **small & local**; let classical stack do global representation & classification.
- QCNNs specialize this for structured data (images / sequences) by **sliding** a small PQC filter across local patches, sharing parameters, and pooling — exactly the conv→pool recipe but with PQCs.

1) What is a QCNN?

QCNNs borrow three CNN principles:

1. **Local receptive fields.** Work on small neighborhoods (e.g., 2×2 pixel → 4 qubits).
2. **Parameter sharing.** Same unitary/filter $U(\theta)$ slides over all patches (weight tying).
3. **Downsampling / pooling.** Reduce spatial size by measuring/aggregating or by unitary pooling + discard.

Basic QCNN block (per patch)

$$\text{patch } x_p \xrightarrow{U_\phi(x_p)} |0\rangle^{\otimes q} \xrightarrow{U(\theta)} |\psi_p\rangle \xrightarrow{\text{measure}} z_p$$

- U_ϕ: encoding (angle, ZZ, or amplitude)
- $U(\theta)$: shared filter (trainable)
- z_p: feature vector per patch (expectations or probabilities)

2) Quanvolutional filters (the "quantum convolution")

2.1 Patch → qubits (encoding)

- **Angle encoding (practical).** Map pixel(s) x to rotations, e.g. $R_Y(\alpha x)$. Scale inputs so $\alpha x \in [-\pi, \pi]$.
- **Entangling map:** Add local ZZ/controlled gates inside the patch to capture interactions. `zz_feature_map` is a simple function builder for this.
- **Amplitude encoding:** packs many dims into $\log_2$ qubits but costly state prep — avoid unless tiny and pre-normalized.
- **Data re-uploading:** interleave data encoding with trainable layers to increase expressivity with few qubits.

2.2 Filter circuit $U(\theta)$

- Use **hardware-efficient** shallow blocks: RY/RZ rotations + CX entanglers implemented via `real_amplitudes` or `efficient_su2` (Qiskit 2.x function builders).
- Keep **reps** small (1–2) on NISQ.

2.3 Decode (measure)

- **Expectation-based:** $z_{p,i} = \langle Z_{q_i} \rangle$ — stable, low-shot.
- **Probability-based:** return bitstring histogram → richer features but much higher shot cost.
- **Fixed projection:** e.g., single decision qubit per patch.
- Output shape = (#patches) × (#features per patch).

3) Parameter sharing (quantum weight tying)

- Implemented by **reusing the same** `ParameterVector('θ', k)` across all patch circuits. This gives one set of trainable weights θ shared by every patch evaluation.
- Benefits: drastically fewer trainable parameters → better generalization and translation equivariance.
- Implementation note: build one parametric circuit and either rebind data per patch or use the same circuit but call the primitive many times with different input bindings.

Example: see binding code in Section 7 (full template).

4) Pooling for QCNNs

4.1 Measurement pooling (practical)

- Keep a subset of qubits per patch (e.g., measure one "decision" qubit).
- Or compute classical aggregates over expectations (mean, max) of patch qubits. Low cost and robust to noise.

4.2 Unitary pooling (research)

- Apply small controlled unitaries across patch qubits then discard one qubit (simulate partial trace). Learns pooling operation but adds two-qubit gates (noise sensitive).
- Use if hardware fidelity allows and you want learnable pooling.

Downsample ratio ≈ (#kept qubits per patch) / (#qubits per patch). Choose pooling to balance feature richness vs. downstream compute.

5) QCNN architectures

Minimal QCNN (4×4 images, 2×2 patches)

- **Quanv layer:** 2×2 patch → 4 qubits → filter → output 1 expectation per patch (stride 2).
- **Pooling:** keep that 1 scalar → 4 features (for a 4×4 image).
- **Head:** classical MLP, e.g., `Linear(4→16) → ReLU → Linear(16→1)` with BCE for binary.

Stacked design (still NISQ-friendly)

- `Quanv(2×2) → Pool → Quanv(2×2 on pooled) → Pool → Dense head`
- Keep **depth ≤ 2 quanv layers** and small patch sizes.

Architectural decisions

- Patch size vs. qubit budget: 2×2 patches map naturally to 4 qubits.
- Stride: starting with stride = patch size minimizes compute (no overlap). Overlap gives more features but multiplies PQC evaluations.

6) Training QCNNs

Losses

- Binary: BCE (or BCEWithLogits)
- Multi-class: cross-entropy
- Regression: MSE (rescale if necessary)

Optimisers & gradients

- **Gradient-based:** Adam / L-BFGS + **parameter-shift** (exact for many rotations). Cost: 2 evaluations per parameter per gradient step.
- **Gradient-free:** SPSA — 2 evaluations/step regardless of p; very useful early in noisy regimes.

Batching & throughput

- **Patch batching:** For fixed θ you can evaluate many patches or many images in a single primitive run — **batch inputs** to amortize latency.
- **Caching:** reuse transpiled circuits (bind parameters) or use Runtime sessions for hardware.

Regularisation & trainability

- L2 on θ, early stopping, small-variance initialization (e.g., $\mathcal{N}(0, 0.01)$).
- **Barren plateau** mitigation: shallow reps, local per-patch cost terms, structured init, feature selection / patch normalization.

7) Qiskit ML implementation patterns (detailed, Qiskit 2.x)

> Use function circuit builders (`zz_feature_map` , `real_amplitudes`), `EstimatorQNN` / `SamplerQNN` , and `TorchConnector` . For hardware, route runs through **Runtime V2** primitives (`EstimatorV2` / `SamplerV2`) and sessions.

7.1 Expectation-based quanv: full PyTorch example (vectorised for speed)

This template shows:

- building one shared filter (ParameterVector θ),
- encoding input patches via a parameter vector `phi` (per-patch),
- wrapping with `EstimatorQNN` and `TorchConnector`,
- applying the quanv filter to all patches in a batch in a vectorized way.

```python
# pip install qiskit qiskit-aer qiskit-machine-learning torch scikit-learn
import numpy as np
import torch, torch.nn as nn
from qiskit import QuantumCircuit
from qiskit.circuit import ParameterVector
from qiskit.circuit.library import zz_feature_map, real_amplitudes
from qiskit_aer.primitives import Estimator
from qiskit_machine_learning.neural_networks import EstimatorQNN
from qiskit_machine_learning.connectors import TorchConnector
```

```
# 1) Build shared parametric filter for 2x2 patches (4 qubits)
q_per_patch = 4
# Data parameters (phi) - one parameter per qubit for angle encoding
phi = ParameterVector("x", length=q_per_patch)
# Shared trainable parameters
theta = ParameterVector("θ", length=6)   # choose p=6 as an example
```

```
# Build encoder circuit using zz_feature_map function and remap parameters to 'p
# We will create a custom small encoder to control parameter naming exactly
enc = QuantumCircuit(q_per_patch)
for i in range(q_per_patch):
    enc.ry(phi[i], i)   # angle encode each pixel
```

```python
# Trainable ansatz: use real_amplitudes (function builder)
# NOTE: real_amplitudes returns its own ParameterVector; to share 'theta' we will
# build a small ansatz manually and use 'theta' as the rotation angles.
ans = QuantumCircuit(q_per_patch)
# simple hardware-efficient block using theta indices
# e.g., RY(theta[0..3]) on each qubit + a chain of CXs + RZ(theta[4..5]) on some qubits
for i in range(q_per_patch):
    ans.ry(theta[i%len(theta)], i)  # simple reuse pattern for small example
for i in range(q_per_patch - 1):
    ans.cx(i, i+1)

qc_patch = enc.compose(ans)  # full patch circuit: encode -> ansatz
```

```python
qc_patch = enc.compose(ans)  # full patch circuit: encode -> ansatz

# 2) Observable: measure Z on qubit 0 (one scalar per patch)
from qiskit.quantum_info import SparsePauliOp
obs = SparsePauliOp.from_list([("Z" + "I"*(q_per_patch-1), 1.0)])

# 3) EstimatorQNN: input_params are phi, weight_params are theta
qnn = EstimatorQNN(
    circuit=qc_patch,
    input_params=list(phi),
    weight_params=list(theta),
    observables=[obs],
    estimator=Estimator(shots=1024)
)
# TorchConnector returns an nn.Module: input shape [B, q_per_patch] -> output [B, 1]
QuanvLayer = TorchConnector(qnn)
```

```python
# 4) Helper: image -> patches (non-overlapping 2x2 stride=2)
def image_to_patches(img_2d):
    """img_2d: numpy array shape (H,W), H and W divisible by 2."""
    patches = []
    H, W = img_2d.shape
    for r in range(0, H, 2):
        for c in range(0, W, 2):
            blk = img_2d[r:r+2, c:c+2].reshape(-1)
            patches.append(blk)
    return np.stack(patches)  # shape [P, 4]
```

```python
# 5) Hybrid model: apply QuanvLayer to each patch (vectorized)
class QCNNHead(nn.Module):
    def __init__(self, quanv_layer, num_patches):
        super().__init__()
        self.quanv = quanv_layer      # Torch module
        self.head = nn.Sequential(nn.Linear(num_patches, 16),
                                  nn.ReLU(),
                                  nn.Linear(16, 1))  # binary output
    def forward(self, x_imgs):  # x_imgs: torch tensor shape [B, H, W]
        B
        fe
        # Vectorize patches across the batch: create tensor shape [B*P, q_per_patch]
        all_patches = []
        for img in x_imgs:
            patches = image_to_patches(img.detach().cpu().numpy())
            all_patches.append(patches)
        all_patches = np.stack(all_patches)    # [B, P, 4]
        B, P, _ = all_patches.shape
        patches_flat = torch.tensor(all_patches.reshape(B*P, -1), dtype=torch.float32)
        # Run quantum layer in batch: QuanvLayer accepts [B*P, q]
        q_out = self.quanv(patches_flat)       # returns [B*P, 1]
        q_out = q_out.view(B, P)               # reshape to [B, P]
        return self.head(q_out)                # [B, 1]
```

```python
# 6) Instantiate model for a 4x4 input (P = 4 patches)
model = QCNNHead(QuanvLayer, num_patches=4)
```


```python
# 6) Instantiate model for a 4x4 input (P = 4 patches)
model = QCNNHead(QuanvLayer, num_patches=4)
```

Notes on efficiency

- The above vectorization creates a batch of patches and runs the quantum layer once over all patches. Under the hood `EstimatorQNN` + `TorchConnector` will call the primitive in batch mode where possible.
- On **hardware**, prefer using `EstimatorV2` with a `run([...])` list of PUB tuples, or use Qiskit Runtime sessions to reduce round–trips. Bindings for each patch are passed as separate parameter sets.

7.2 Probability-based quanv (SamplerQNN)

When you want bitstring histograms per patch (richer but heavier):

- Use `Sampler` (local sim) or `SamplerV2` (Runtime) to get distributions.
- Convert distributions to features: e.g., indicator sums, parity features, or probabilities of a subset of bitstrings.
- `SamplerQNN` + `TorchConnector` can map bitstring probs to tensors.

Tradeoffs: full-histogram → 2^q outputs per patch (explodes with q). For q=4, $2^4 = 16$ outputs per patch — expensive but sometimes useful.

8) Evaluation & when QCNNs help

Metrics to report

- Accuracy / F1 vs classical baseline (small CNN / logistic).
- Parameter count (QCNN + head vs small CNN).
- Shot budget & wall-time (circuit evals per epoch).
- Robustness under injected noise.

When QCNNs can shine today

- Tiny images (4×4, 8×8 crops), few-shot regimes, class-imbalanced datasets, or as part of hybrid feature extractor pipelines.

9) Practical tips & pitfalls

- **Scale inputs** to angle range $[-\pi, \pi]$.
- **Stride = patch size** to start — avoids explosion of PQC calls. Consider overlap only if compute budget allows.
- **Keep filters shallow** (1–2 reps) to avoid barren plateaus / noise.
- **Topology-aware entanglement**: map qubits to device coupling to avoid SWAPs.
- **Batch patches** (vectorize) and reuse transpiled circuits or use Runtime sessions.
- **Readout mitigation**: invert confusion matrices to debias $\langle Z \rangle$.
- **Prefer** `EstimatorQNN` / `SamplerQNN` (Qiskit ML) + `TorchConnector` for production prototypes; avoid retired `CircuitQNN`/Opflow patterns.

10) Worked example — conceptual pipeline (step-by-step)

1. **Dataset:** downsample MNIST to 4×4 (or crop 8×8). Normalize into $[0, 1]$.
2. **Preprocess:** map pixels via angle: $\alpha x \mapsto R_Y(\alpha x)$ with $\alpha = \pi$.
3. **Quanv:** 2×2 patch → 4 qubits; `ry` encoding + small ansatz → measure `Z0` per patch.
4. **Pooling:** keep one scalar per patch → e.g., 4 features for 4×4 input.
5. **Head:** MLP `Linear(4→16)→ReLU→Linear(16→10)` (for 10-way) or `→1` + BCE.
6. **Optimiser:** SPSA early; Adam with parameter-shift for fine-tuning. Shots: start 512 → 2048 near convergence.
7. **Ablations:** compare to small classical conv with same receptive field & parameter count.

13) Practical engineering checklist (copyable)

- [ ] Input pixels normalized to $[-\pi, \pi]$ for angle encodings.
- [ ] Patch size chosen (2×2 recommended for q=4).
- [ ] Shared `ParameterVector('θ')` used across patches.
- [ ] Vectorized patch batching in forward pass to reduce calls.
- [ ] Shots schedule: start 256–512, raise to 2k at fine-tune.
- [ ] Readout mitigation enabled for hardware runs.
- [ ] Topology-aware entanglement & initial layout used.
- [ ] Baseline classical model implemented for fair comparison.
- [ ] Ablations: reps, shots, stride, pooling type recorded.